Donanımsal Senkronizasyon Çözümleri (Hardware Synchronization) - Sınav Hazırlık Özeti

Bu bölüm, yazılımsal çözümlerin yetersiz kaldığı veya performans sorunu yarattığı yerlerde, **doğrudan işlemci (CPU) ve bellek mimarisi seviyesinde** sunulan donanımsal çözümleri inceler.

Hocalar bu bölümden özellikle **"Hangi çözüm hangi sistemde işe yarar/yaramaz?"** tarzı karşılaştırma soruları ile **Bounded-Waiting TestAndSet** kodunu sormayı çok severler.

---

1. Kesmeleri Kapatmak (Interrupt Disable) - "Balyoz Yöntemi"

Bir proses Kritik Bölge'de (CS) çalışırken, işlemcinin dışarıdan gelen (saat kesmesi vb.) tüm uyarıları (interrupts) dinlemeyi reddetmesidir. İşlemci adeta "kapıyı kilitler".

- **Nasıl Çalışır?** İşlemci, kesmeleri kapattığında hiçbir şekilde onu yarıda kesip (preempt edip) bağlam değişimi (context switch) yapamaz. Bu yapı **Non-preemptive** (araya girilemez) mantığı benimser.

🚨 Sınav Sorusunun Geleceği Yer: Single-Processor vs Multi-Processor

Sistem Türü	Kesmeleri Kapatma (Interrupt Disable) İşe Yarar mı? Neden?
Tek İşlemcili (Single Processor)	✅ **Evet, harika çalışır.** Tek bir işlemci olduğu için, o işlemciyi dış dünyaya kapatmak başka hiçbir kodun araya girmemesini (Mutual Exclusion) kesin olarak sağlar.
Çok İşlemcili (Multi-processor)	❌ **Uygun Değildir.** Kesmeleri kapat emrini diğer TÜM işlemcilere / çekirdeklere mesaj olarak göndermek inanılmaz **vakit alır (gecikme yaratır)**. Her CS girişinde bu yapıldığı için sistem verimliliği ve hızı (efficiency) yerle bir olur.

---

2. Bellek Modelleri ve Bariyerleri (Memory Barriers)

Farklı işlemciler, performansı artırmak adına arka planda komutların sırasını (instruction reordering) değiştirebilir. Bu, özellikle ortak bellek kullanan thread'ler arasında ölümcül hatalar doğurur.

A) Bellek Modelleri (Memory Models)

1. **Strongly Ordered (Güçlü Sıralı):** Bir işlemcinin bellekte yaptığı değişiklik, diğer tüm işlemciler tarafından **ANINDA (hemen)** görülür.
2. **Weakly Ordered (Zayıf Sıralı):** Bir işlemcinin bellek değişikliği, diğerleri tarafından **hemen görülmeyebilir**.

B) Memory Barrier (Bellek Bariyeri) Nedir?

Bellekteki bir değişikliğin **diğer tüm işlemcilere yayılmasını ve görünür kılınmasını ZORLAYAN** özel donanımlara has komuttur.

> 💡 ***Kod Örneğinin Mantığı (Hoca Ne Demek İstiyor?):*** *Görseldeki* `x = 100` *ve* `flag = true` *örneğinde, zayıf sıralı (weakly ordered) bir işlemci hız kazanmak için* `flag = true` *işlemini hafızaya daha önce yazabilir. Eğer Thread 1,* `flag` *'in true olduğunu görüp döngüden çıkar ve* `x` *'i ekrana basarsa (ama* `x` *henüz* `100` *güncellenmediyse) HATA ÇIKAR. Bu yüzden araya* `memory_barrier();` *konur. Bu komut der ki: "Önce üstteki* `x=100` *'ü belleğe kesin olarak yaz, o load/store bitmeden ASLA alt satıra (* `flag=true` *) geçme!"*

> ***Not:*** *Memory Barrier çok düşük seviyeli bir donanım kodudur, genelde sıradan programcılar değil,* ***İşletim Sistemi (Kernel) geliştiricileri*** *Mutual Exclusion sağlamak için kullanır.*

---

3. Atomik Özel Donanım Komutları (Hardware Instructions)

İşletim sistemleri, Kritik Bölge problemini aşmak için yürütülmesi sırasında **ASLA kesintiye (interrupt'a) uğramayan, "Atomik" (bölünemez)** özel donanım komutları sunar. Multiprocessor bir sistemde iki çekirdek aynı anda bu komutu çağırsalar dahi, donanım onları araya sadece biri girecek şekilde tek tek alır.

İki Soyut Donanım Komutu:

1. `test_and_set()` : İçeriği (genelde byte/word) okur ve hemen ardından değerini true (kilitleme) yapar.
2. `compare_and_swap()` (CAS) : İki veriyi atomik şekilde karşılaştırır, eşitse yerlerini değiştirir.

> ❓ ***Hoca Testte Sorabilir (Tuzak Soru):*** *`TestAndSet` ve `CompareAndSwap` donanımsal özellikleri sayesinde tek başlarına CS'nin 3 altın kuralını da sağlar mı?* ✅ ***Cevap:*** *HAYIR!* ***Mutual Exclusion (Karşılıklı Dışlama)****'yı ve ilerlemeyi çok iyi sağlarlar. Ancak saf (sade) halleriyle kullanıldıklarında* ***Bounded-Waiting (Sınırlı Bekleme)*** *şartını sağlayamazlar, Starvation (Açlık) doğurabilirler.*

---

4. 🚨 GÖRSELDEKİ KODUN DEŞİFRESİ: "Bounded-Waiting with TestAndSet"

(İşte hocanın slaytta "Bu karmaşık kod nə yapar???" diye vuracağı kısım tam olarak burası!)

Eklediğiniz görselde karmaşık gibi duran, tablolu bir **"Bounded wait solution with TestAndSet"** kodu var. Hoca "Mutex exclusion sağlıyor mu??" diye not düşmüş.

Soru 1: Bu kod NEDEN bu kadar uzadı? Sade TestAndSet yetmiyor muydu?

Yukarıda dediğimiz gibi: Sade bir `TestAndSet` kodu, iki prosesten sadece birisinin içeri girmesine izin verir **(Mutual Exclusion SAĞLAR)**. İlerlemeyi de sağlar. ❌ **ANCAK, Bounded-Waiting (Sınırlı Bekleme) ŞARTINI SAĞLAYAMAZ!** Kapıda bekleyen P0, P1, P2, P3 varken, P0 çıkıp CS'yi açtığında kimin daha hızlı davranıp içeri gireceği tamamen donanım hızında rastgeledir. Şanssız bir proses "Açlık (Starvation)" yaşayabilir, sonsuza dek içeri giremeyebilir.

✅ **Çözüm (Görseldeki Kod):** Starvation'u önlemek ve "Sınırlı Bekleme" kuralını sağlamak için koda `waiting[n]` (bekleyenler) isimli bir dizi (array) eklenmiştir.

Soru 2: Entry Section (Giriş) Adım Adım Ne Yapıyor?

Görseldeki `while` döngüsü olan kısım:

```
waiting[i] = true;             // Adım 1: "Ben (Pi) kapıya geldim, içeri girmek için
kuyruktayım" (Bayrak Kaldır).
key = true;                    // Adım 2: Elimize sanal bir kilit anahtarı (key) alıyoruz.
while (waiting[i] && key)      // Adım 3: Pi hem "bekliyor" pozisyonundaysa hem de "anahtar
henüz bende" değilse dönmeye (spin) devam et.
    key = TestAndSet(&lock);   // Adım 4: KİLİDİ ZORLA. Eğer açık (lock=false) ise
TestAndSet onu (true) kilitler ve kendisi false döner! false dönerse while kırılır!
waiting[i] = false;            // Adım 5: Artık kilidi kırdım içeriye giriyorum.
"Bekliyorum" bayrağımı false (F) yaparak CS'ye daldım.
```

```
// ---> BURASI KRİTİK BÖLGE (CS) <---
```

Görseldeki tablonun açıklaması: P0, P1, P2 gibi prosesler var. Tabloda P0 kapıya gelip `waiting[0]=T` yapmış, `key=T` ayarlamış ve TestAndSet kapısını dövmektedir. Başardığında `waiting[0]` tablosu `F` olacak.

Soru 3: Exit Section Algoritması Nasıl Adalet Sağlıyor? (Çıkış Kısmı)

Bencilce çıkıp `lock = false` yapıp kaçılmaz. Bir j prosesi çıkarılır (i+1 mod n şeklinde). Eğer kapıda `waiting[j] == true` olan biri varsa, direk onun `waiting[j]` değerini false yapar! Bu sayede onu döngüden manuel olarak kurtarıp kilidi bizzat ona hediye etmiş olur. İşte **Sınırlı Bekleme (Bounded Waiting)** garanti altına alınmış olur!

> 🔥 ***En Net Cevap ("Mutex exclusion sağlıyor mu??" notuna):*** *Evet kesinlikle sağlıyor. Çünkü `TestAndSet(&Lock)` fonksiyonu donanım tabanlı Atomik (Bölünemez) bir fonksiyondur. İki proses P0 ve P1 döngü içerisinde aynı milisaniyede `Lock`'a saldırsalar bile, işlemci donanımı yalnızca P0'a `false` döner ve hemen kapıyı kapatır. P1 `true` okuyarak kapıda beklemeye devam eder. O yüzden* ***Mutual Exclusion Kesinlikle Sağlanır****.*

🎯 5. SINAV PROVASI: KELİME AVI VE TESPİT SORULARI

1. Boşluk Doldurma: Bir değişkenin belleğe yazılması ve diğer işlemciler tarafından da görülmesinin arka planda gecikmeli olarak sağlanabildiği bellek mimarilerine **[Weakly Ordered (Zayıf Sıralı)]** bellek modeli denir. Yazılım ve donanım senkronizasyon kaymasını engellemek ve işlem sırasını zorunlu tutmak için **[Memory Barrier (Bellek Bariyeri)]** komutları kullanılır.

2. Klasik Doğru/Yanlış Yanıltmacası:

- *"Tek işlemcili bir sistemde (Single Processor), dış kesmeleri (interrupt) kapatmak Kritik Bölge problemini çözmek için harika ve tam garantili (Non-preemptive olduğu için) bir yoldur."* -> **DOĞRU!** Başka hiçbir komut/görev (timer dahil) araya giremeyeceği için CS güvenlidir.
- *"Mükemmel çalıştığı için, Çok çekirdekli (Multiprocessor) sistemlerde de kesmeleri kapatmak en iyi çözümdür."* -> **YANLIŞ!** Multi-process sistemlerde kesmeleri kapattırma sinyalini tüm çekirdeklere iletmek zaman alır, aşırı performans düşüşü yaratır.

3. Görselle Alakalı / Kod Klasik Senaryo Sorusu: *Soru:* İşletim sistemlerinde sunulan `TestAndSet` veya `CompareAndSwap` donanım talimatlarının tek başlarına kullanıldığında yetersiz kaldığı/çözemeyebileceği Kritik Bölge şartı hangisidir? Yukarıdaki görselde bu eksiği kapatmak için nasıl bir yaklaşım sergilenmiştir? *Cevap:* Atomik donanım talimatları tek başlarına **Bounded Waiting (Sınırlı Bekleme)** kuralını garanti edemez ve **Starvation (Açlığa)** yol açıp bir prosesin sonsuza dek kuyrukta beklemesine neden olabilirler. Görseldeki çözümde bu boşluğu kapatmak için, koda kapıda bekleyenleri adaletli sıraya koyan `waiting[i]` boolean dizisi eklenmesi yaklaşımı sergilenmiştir. Bu sayede çıkış yapan proses, bencilce kilidi fırlatıp atmak yerine kendinden sonra sırası gelen ilk bekleyen prosese kilidi bizzat teslim eder.Donanım Seviyesinde (Hardware) en kaba ama en etkili çözüme geldik. Bu yöntem, özellikle tek işlemcili (single-processor) sistemlerde "balyoz" etkisi yaratır.

1. Konuyu Açıkla: "Telefonu Kablosundan Çekmek" Bu yaklaşımı şöyle düşün: Bir odada (Kritik Bölge) çok önemli bir iş yapıyorsun ve biri içeri girip işini bölerse her şey mahvolacak. O yüzden, o odada çalışırken kapıyı kilitleyip (Interrupt Disable) kimsenin içeri girmesine izin vermiyorsun. İşin bitince kapıyı açıyorsun.

Single processor: preempted olmasına ızıen vermez ve context swıcth gercekleşmez

Nonprempted tarafından benımsenmıstır

Teknik Karşılığı:

Interrupt Disable (Kesmeleri Kapat): İşlemciye "Ben açana kadar hiçbir dış uyarıyı, saati veya başka bir süreci dinleme" dersin.

No Preemption: İşlemci seni yarıda kesip (preempt) başka bir sürece geçemez (context switch yapamaz). Çünkü işlemcinin "sıradaki sürece geçme vaktin geldi" diyen iç saati (timer interrupt) bile susturulmuştur.

Single processor sistemler için önerilen critical seciton problemi çözümü multiprocessor sistemler için uygun çözüm değildir. □ multiprocessor sistemlerde interrupt'larındisable/enable yapılmasıiçin tümişlemcileremesaj göndermek zaman alıcı olabilir. Bu mesajın iletilmesi, her bir critical sectiona girişi geciktirir ve sistemin verimliliği düşer.

Memory model, bir bilgisayar mimarisinin uygulama programlarına yaptığı bellek garantileridir. □ Memory model türleri: □ Strongly ordered –bir işlemcideki bellek değişikliği diğer tüm işlemciler tarafından hemen görülebilir. □ Weakly ordered –bir işlemcidebellekte yapılandeğişikliklerdiğer işlemcilertarafındanhemen görülmeyebilir. □ Memory barrier'lerbellekteki herhangi bir değişikliğin diğer tüm işlemcilere yayılmasını (görünür kılınmasını) zorlayan komutlardır. Memory Barrier

Memory barrier'ler, bir işlemcideki bellek değişikliklerinin diğer tüm işlemcilere yayılmasını (görünür kılınmasını) zorlayan komutlardır. Bu, özellikle weakly ordered memory model kullanan sistemlerde önemlidir. Memory barrier'ler, kritik bölgeye girmeden önce ve çıktıktan sonra kullanılarak, bellek tutarlılığını sağlar ve veri yarışlarını önler.

Memory barrier'lerçok düşük seviyeli işlemlerdir ve genellikle yalnızca kernel geliştiriciler tarafından mutual exclusion'ısağlamak için yazılan özel kodlarda kullanılmaktadır. Memory Barrier 11

Örnekte thread 1 çıkışının 100 olmasını sağlamak için aşağıdaki memory barrier komutları ekleyebiliriz. □ Thread 1 while (!flag) memory_barrier(); print x □ Thread 2 x = 100; memory_barrier(); flag = true □ Böylece thread 1 için flag değerinin x değerinden önce load edilmesi ve thread 2 için x = 100 komutunun flag = true komutundan önce gerçekleşmesi garanti edilir. Memory Barrier 10

Memory barrier'lerçok düşük seviyeli işlemlerdir ve genellikle yalnızca kernel geliştiriciler tarafından mutual exclusion'ısağlamak için yazılan özel kodlarda kullanılmaktadır. Memory Barrier 11

hardware instructions

13

90% □ Single processor sistemler için önerilen critical seciton problemi çözümü multiprocessor sistemler için uygun çözüm değildir. □ multiprocessor sistemlerde interrupt'larındisable/enable yapılmasıiçin tümişlemcileremesaj göndermek zaman alıcı olabilir. Bu mesajın iletilmesi, her bir critical sectiona girişi geciktirir ve sistemin verimliliği düşer. □ Bu bölümde, CS probleminin çözmek için destek sağlayan üç donanım komutu sunulacaktır: □ Memory barriers □ Hardware instructions □ Atomic variables Synchronization Hardware 5 □ Memory barriers □ Hardware instructions □ Atomic variables Synchronization Hardware 6 □ Memory model, bir bilgisayar mimarisinin uygulama programlarına yaptığı bellek garantileridir. □ Memory model türleri: □ Strongly ordered –bir işlemcideki bellek değişikliği diğer tüm işlemciler tarafından hemen görülebilir. □ Weakly ordered –bir işlemcidebellekte yapılandeğişikliklerdiğer işlemcilertarafındanhemen görülmeyebilir. □ Memory barrier'lerbellekteki herhangi bir değişikliğin diğer tüm işlemcilere yayılmasını (görünür kılınmasını) zorlayan komutlardır. Memory Barrier 7 □ Bir memory barrier komutu gerçekleştirildiğinde, sistem sonraki herhangi bir load veya store işlemi gerçekleştirilmeden önce tüm load ve store işlemlerinin tamamlandığından emin olur. □ Bu nedenle, komutlar yeniden sıralansa bile memory barrier, store işlemlerinin bellekte tamamlanmasını ve gelecekteki load veya store işlemleri gerçekleştirilmeden önce diğer işlemciler tarafından görülmesini sağlar. Memory Barrier 8 ■ Örnek: İki thread tarafından paylaşılan veriler Thread1 Thread2 Memory Barrier 9 □ Örnekte thread 1 çıkışının 100 olmasını sağlamak için aşağıdaki memory barrier komutları ekleyebiliriz. □ Thread 1 while (!flag) memory_barrier(); print x □ Thread 2 x = 100; memory_barrier(); flag = true □ Böylece thread 1 için flag değerinin x değerinden önce load edilmesi ve thread 2 için x = 100 komutunun flag = true komutundan önce gerçekleşmesi garanti edilir. Memory Barrier 10 □ Peterson'ınçözümüyle ilgili olarak daha önce gösterilen turn ve flag atamalarının yeniden sıralanması problemini

önlemek için giriş bölümündeki flag ve turn atamaları arasına bir memory barrier yerleştirebilir. Memory barrier'lerçok düşük seviyeli işlemlerdir ve genellikle yalnızca kernel geliştiriciler tarafından mutual exclusion'ısağlamak için yazılan özel kodlarda kullanılmaktadır. Memory Barrier 11 Memory barriers Hardware instructions Atomic variables Synchronization Hardware 12 Process senkronizasyonu için diğer bir donanım mekanizması: yürütülmesi sırasında interrupt olmayan atomic olarak yürütülen özel donanım komutları (hardware instructions) kullanmaktır. Multiprocess sistemlerde bu özel komutlar aynı andan yürütülmeye çalışıldığında bunlar sırasıyla yürütülür, birinin işi tamamlanmadan araya başkası giremez. Hardware Instructions 13 Bir word (2 byte -16bitlik veri) içeriğinitest edip değiştirme veya iki ayrı word içeriğiniyer değiştirme(swap) işlemlerini atomik (interrupta müsaade edilmeden tek bir seferde yürütülen birim)olarak yapan özel donanımkomutları, critical section problemini basit bir şekilde çözmek için kullanılabilir. Belirli bir makine için belirli donanımsal komut isimleri yerine test_and_set() ve compare_and_swap() komutları soyut komut isimleri olarak bu bölümde kullanılmıştır. Bu iki soyut komut zaten çoğu mimaride varolan mimariden mimariye isimleri değişebilen ancak yaptıkları iş itibariyle mantıksal olarak bu soyut komutlarla aynı işi yapan komutları temsil etmektedir. Bu iki soyut komut, critical section probleminin donanım desteği ile nasıl çözülebileceğini göstermek için kullanılacaktır. Hardware Instructions

Process senkronizasyonu için diğer bir donanım mekanizması: yürütülmesi sırasında interrupt olmayan atomic olarak yürütülen özel donanım komutları (hardware instructions) kullanmaktır. Multiprocess sistemlerde bu özel komutlar aynı andan yürütülmeye çalışıldığında bunlar sırasıyla yürütülür, birinin işi tamamlanmadan araya başkası giremez. Hardware Instructions 13

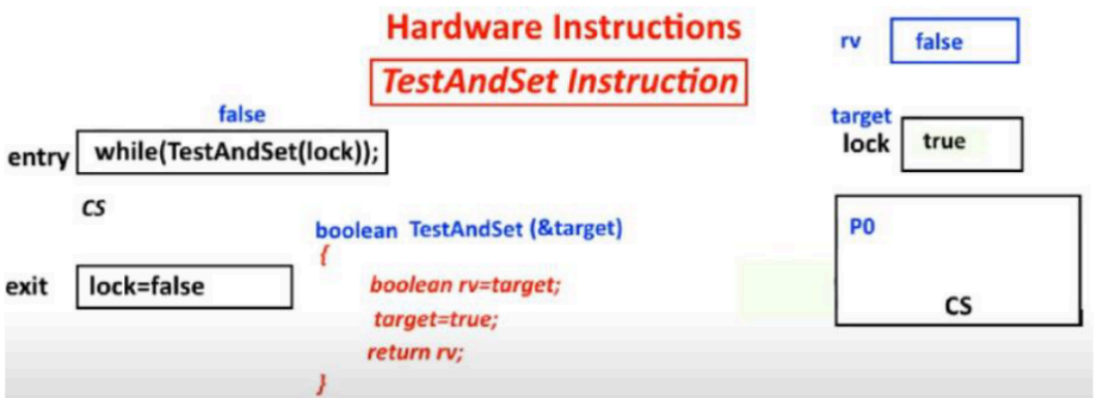
```
do {
    acquire lock
            critical section
    release lock
            remainder section
} while (TRUE);
```

test_and_set() Kullanımıyla Mutual Exclusion Gerçekleştirimi

Boolean türündeki **lock** değişkeni processler arasında paylaşılır, başlangıç değeri FALSE olarak atanır.

```
do {
   while (test_and_set(&lock))
      ; /* do nothing */

      /* critical section */

   lock = false;

      /* remainder section */
} while (true);
```

```
boolean test_and_set(boolean *target) {
   boolean rv = *target;
   *target = true;

   return rv;
}
```

Mutex exculıon saglıyor mu???

- **Mutual exclusion sağlanıyor mu?**